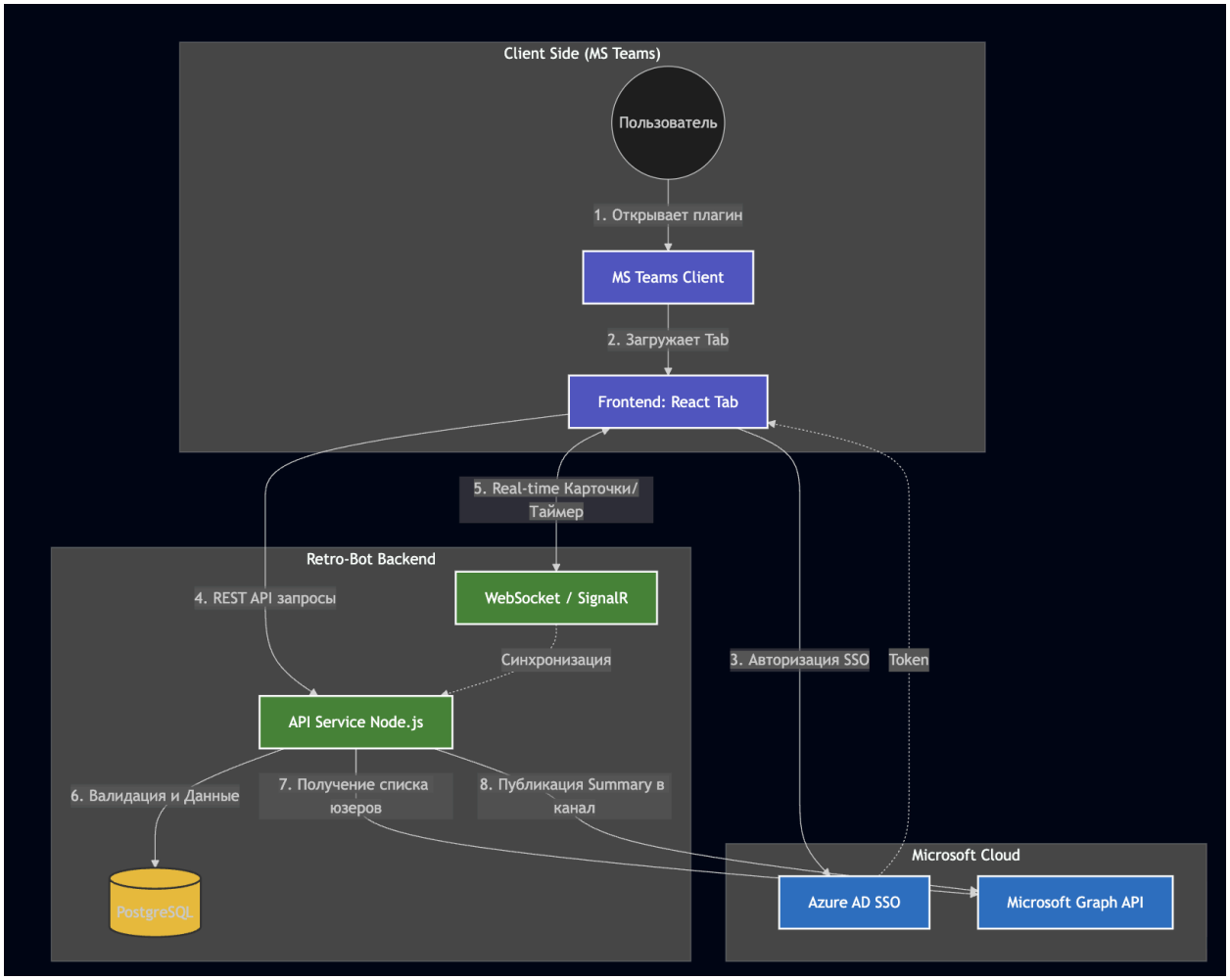
Схема взаимодействия (Data Flow)

1. **Клиент (Teams Tab):** Инициализирует сессию, получает Context (User ID, Channel ID, Team ID) через Teams SDK.
2. **Auth Middleware:** Проверяет JWT токен через Azure AD.
3. **Backend:** Обрабатывает бизнес-логику (создание карточек, голосование, таймер).
4. **WebSocket Gateway:** Рассылает обновления (`CARD_ADDED`, `PHASE_CHANGED`) всем подключенным клиентам в реальном времени.



Модель данных (Database Schema)

ER-Diagram Description

`sessions`

Хранит состояние ретроспективы.

SQL

```sql
CREATE TABLE sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    ms_team_id VARCHAR(255) NOT NULL,    -- Tenant ID команды
    ms_channel_id VARCHAR(255) NOT NULL, -- ID канала
    status VARCHAR(50) DEFAULT 'ACTIVE', -- ACTIVE, COMPLETED, ARCHIVED
    current_phase VARCHAR(50) DEFAULT 'COLLECT', -- COLLECT, VOTE, SUMMARY
    template_type VARCHAR(50) NOT NULL,  -- START_STOP, 4L, MAD_SAD
    timer_expires_at TIMESTAMP,          -- Время окончания фазы
    created_at TIMESTAMP DEFAULT NOW(),
    report_message_id VARCHAR(255)       -- ID сообщения с отчетом в Teams
);
```

`cards` (Анонимные карточки)

Здесь реализуется приватность. Мы не храним `user_id`.

SQL

```sql
CREATE TABLE cards (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    session_id UUID REFERENCES sessions(id) ON DELETE CASCADE,
    column_key VARCHAR(50) NOT NULL,     -- 'mad', 'sad', 'glad'
    content TEXT CHECK (length(content) <= 500), -- Ограничение по ТЗ

    -- HASH = SHA256(User_Azure_OID + Session_Secret_Key)
    -- Позволяет пользователю редактировать свое, но не раскрывает его
личность админу БД.
    owner_hash VARCHAR(255) NOT NULL,

    votes_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);
```

`votes`

Таблица для контроля лимитов голосования .

```SQL
CREATE TABLE votes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    card_id UUID REFERENCES cards(id) ON DELETE CASCADE,

    -- HASH = SHA256(User_Azure_OID + Session_ID)
    voter_hash VARCHAR(255) NOT NULL
);
```

API Контракты (REST + WebSocket)

REST API (Основные операции)

POST /api/sessions (Создание сессии)

- Request: `{ template: "MAD_SAD", duration: 10 }`
- Response: `{ sessionId: "uuid", status: "ACTIVE" }`

POST /api/cards (Добавление карточки)

- Headers: `Authorization: Bearer <Teams_Token>`
- Request: `{ sessionId: "uuid", column: "mad", text: "Slow CI/CD" }`
- Logic: Генерирует хэш, сохраняет, триггерит сокет-событие.

PUT /api/sessions/:id/phase (Смена этапа)

- Request: `{ nextPhase: "VOTE" }`
- Logic: Блокирует создание новых карточек, включает режим голосования.

POST /api/sessions/:id/summary (Публикация отчета)

- Logic: Генерирует Adaptive Card JSON, отправляет в канал через Graph API, закрывает сессию.

WebSocket Events (Real-time)

Event Name	Direction	Payload	Описание

join_session	Client -> Server	{ sessionId }	Подключение к комнате
card_created	Server -> Client	{ cardId, column, text }	Новая карточка на доске
card_deleted	Server -> Client	{ cardId }	Удаление карточки
phase_changed	Server -> Client	{ newPhase }	Принудительное переключение экрана
vote_updated	Server -> Client	{ cardId, newCount }	Обновление счетчика голосов

Флоу

1. Инсталляция и Авторизация (F1)

- **Действие:** Установка плагина в канал MS Teams и бесшовная авторизация через SSO.
- **Сеть:** `POST /api/auth/login` (REST) ➔ Открытие сокета `WS /ws/connect`.
- **База данных:**
  - `Teams` (Создание): Запись `ms_team_id` и `ms_channel_id`.
  - `Users` (Чтение/Создание): Идентификация `azure_oid` и определение роли (`role`).

2. Инициация сессии (F2)

- **Действие:** Модератор настраивает тему, шаблон, таймеры и запускает сессию.

- **Сеть:** `POST /api/sessions` (создание) ➔ `PUT /api/sessions/{id}/status` (смена статуса) ➔ `WS: SESSION_STATUS_CHANGED` (обновление UI у всех).
- **База данных:**
  - `Sessions` (Создание): Запись `title`, `template_type`, `timer_settings`. Установка `status = Draft`.
  - `Sessions` (Обновление): Перевод `status` в `Collection`.

3. Сбор данных (F3)

- **Действие:** Участники анонимно пишут, редактируют или удаляют свои карточки.
- **Сеть:** `POST /api/cards`, `PUT/DELETE /api/cards/{id}` ➔ `WS: CARD_CREATED/UPDATED/DELETED` (мгновенная синхронизация доски).
- **База данных:**
  - `Cards` (Создание/Обновление): Запись `session_id`, `category` (колонка шаблона), `content_text` (до 500 символов).
  - `Cards` (Валидация): Проверка или запись `author_hash` (для редактирования своих карточек без нарушения анонимности).

4. Приоритизация / Голосование (F4)

- **Действие:** Модератор включает голосование. Участники распределяют ограниченный лимит голосов.
- **Сеть:** `PUT /api/sessions/{id}/status` ➔ `POST /api/cards/{id}/vote` ➔ `WS: CARD_VOTE_UPDATED` (обновление счетчиков в лайве).
- **База данных:**
  - `Sessions` (Обновление): `status` меняется на `Voting` (блокирует редактирование карточек).
  - `Votes` (Чтение/Создание): Проверка лимита голосов по `user_hash`, затем создание записи с привязкой к `card_id`.

5. Финализация и Экспорт (F5)

- **Действие:** Формирование структурированного отчета (Summary) и его автоматическая публикация в канал.
- **Сеть:** `GET /api/sessions/{id}/summary` (предпросмотр) ➔ `POST /api/sessions/{id}/publish` (отправка в Teams).
- **База данных:**
  - `Sessions` (Обновление): `status` меняется на `Summary`, затем на `Completed`. Сохранение `summary_link` для вкладки "История".
  - `Cards` + `Votes` (Чтение): Агрегация данных — группировка карточек по `category` (колонкам) и подсчет связанных голосов.